МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ 1

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФеП-23

МАГОРА Максим

Перевірив:

СВАЛЕБА Сергій Андрійович

з дисципліни «Функціональне програмування»

Тема: Основи функціонального програмування у Python

Варіант № 11: Мобільний тариф

1. Мета та завдання роботи

- Зрозуміти сутність чистої функції та референтної прозорості.

  DOCX

- Навчитися виявляти та ізолювати побічні ефекти (I/O, мутації, глобальний стан).

  DOCX

- Переписати імперативний фрагмент коду у функціональному стилі.

  DOCX

- Застосувати typing.Callable для параметризації обчислень, фільтрації та політик бонусів.

  DOCX

- Інтегрувати специфічні ключові слова
  варіанта: mobile, tariff, minutes, sms, traffic, data, bonus, monthly, cost, limit, filter.

2. Структура проєкту та модулів (Варіант № 11)

Проєкт організовано відповідно до вимог методичних вказівок і структуровано у вигляді таких папок та модулів:

lab01/

├── core.py # Чисте функціональне ядро (розрахунки тарифів, політики)

├── app.py # Оболонка побічних ефектів та I/O (інтерфейс виведення)

├── tests/

│ └── test_core.py # Pytest-тести на референтну прозорість та відсутність мутацій

├── requirements.txt # Залежності проєкту (pytest, mypy, black, ruff)

└── README.md # Документація та інструкція із запуску

3. Опис і назви методів (функцій), реалізованих у проєкті

Для Варіанта 11 («Мобільний тариф») реалізовано такі чисті функції та типи:

A. Чисте ядро (core.py)

- **calculate_extra_minutes_cost(record: UsageRecord) -> float**

  Опис: Чиста детермінована функція розрахунку вартості понаднормових хвилин (minutes) понад ліміт (limit).

- **calculate_extra_sms_cost(record: UsageRecord) -> float**

  Опис: Чиста функція обчислення вартості перевищення ліміту повідомлень (sms).

- **calculate_extra_gb_cost(record: UsageRecord) -> float**

  Опис: Чиста функція розрахунку вартості понаднормового інтернет-трафіку/даних (traffic, data, gb).

- **compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]**

  Опис: Функція композиції для побудови складніших правил із простіших блоків.

- **make_loyalty_bonus_policy(discount_percent: float) -> BonusFn**

  Опис: Фабрика функцій для генерації політики знижок/бонусів (bonus, tariff) через Callable.

- **with_monthly_cost(record: UsageRecord, ...) -> ProcessedUsageRecord**

  Опис: Аналог функції with_total з методички — повертає **новий словник** із розрахованими показниками місячної вартості (monthly, cost), не мутуючи вхідний об'єкт.

- **make_tariff_processor(*, accept: FilterFn, bonus_policy: BonusFn) -> Callable[...]**

  Опис: Функція вищого порядку (фабрика обробника), що приймає правила фільтрації (filter, limit) та політики обчислень через typing.Callable.

Б. Оболонка побічних ефектів (app.py)

- **render_report(result: dict[str, object]) -> None**

  Опис: Функція виведення (I/O, print), яка приймає чистий результат роботи ядра та відображає звіт у консолі.

- **run() -> None**

Опис: Точка входу в програму, що ініціалізує початкові дані (mobile, tariff) та запускає обробку.

4. Висновки

Під час виконання лабораторної роботи №1 (варіант 11 — «Мобільний тариф») було практично закріплено принципи функціонального програмування на мові Python:

1. **Чистота функцій та референтна прозорість:** Усі розрахункові функції (calculate_extra_minutes_cost,calculate_extra_sms_cost, calculate_extra_gb_cost) є повністю детермінованими та не залежать від зовнішнього або глобального стану.
2. **Ізоляція побічних ефектів:** Операції введення-виведення (print) винесено до модуля app.py, а заміна мутацій створенням нових об'єктів реалізована через функцію with_monthly_cost.
3. **Параметризація через Callable:** Використання функцій вищого порядку (make_tariff_processor,make_loyalty_bonus_policy) дозволило відокремити бізнес-правила фільтрації та нарахування бонусів від загальної структури обробки даних.
4. **Тестування:** За допомогою модульних pytest-тестів (test_referential_transparency, test_no_mutation, test_callable_policies) успішно перевірено стабільність ядра та відсутність побічних ефектів.

Вихідний код проєкту

1. core.py

"""Чисте функціональне ядро для розрахунку мобільного тарифу (Варіант 11).

Ключові слова: mobile, tariff, minutes, sms, traffic, data, bonus, monthly, cost, limit, filter.

Не містить побічних ефектів (I/O) та не мутує вхідні дані.

"""

from typing import Callable, Iterable, List, TypedDict, TypeVar

--- Анотації типів предметної області ---

class TariffPlan(TypedDict):
    base_fee: float
    free_minutes: int
    free_sms: int
    free_gb: float
    price_per_min: float
    price_per_sms: float
    price_per_gb: float

```
class UsageRecord(TypedDict):
    user_id: int
    minutes_used: int
    sms_used: int
    gb_used: float
    plan: TariffPlan



class ProcessedUsageRecord(UsageRecord):
    extra_minutes_cost: float
    extra_sms_cost: float
    extra_gb_cost: float
    monthly_cost: float



# Синоніми типів для політик Callable
BonusFn = Callable[[float], float]
FilterFn = Callable[[UsageRecord], bool]

A = TypeVar("A")
```

```
B = TypeVar("B")

C = TypeVar("C")


# --- Чисті функції предметної області ---



def calculate_extra_minutes_cost(record: UsageRecord) -> float:
    """Обчислює вартість понаднормових хвилин (minutes) понад ліміт."""
    plan = record["plan"]
    extra_min = max(0, record["minutes_used"] - plan["free_minutes"])
    return extra_min * plan["price_per_min"]



def calculate_extra_sms_cost(record: UsageRecord) -> float:
    """Обчислює вартість понаднормових SMS-повідомлень."""
    plan = record["plan"]
    extra_sms = max(0, record["sms_used"] - plan["free_sms"])
    return extra_sms * plan["price_per_sms"]
```

```
def calculate_extra_gb_cost(record: UsageRecord) -> float:
    """Обчислює вартість понаднормового інтернет-трафіку/даних (traffic, data, gb)."""
    plan = record["plan"]
    extra_gb = max(0.0, record["gb_used"] - plan["free_gb"])
    return extra_gb * plan["price_per_gb"]



def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    """Компоновка двох функцій: (f ∘ g)(x) = f(g(x))."""
    return lambda x: f(g(x))



def make_loyalty_bonus_policy(discount_percent: float) -> BonusFn:
    """Фабрика функцій: повертає політику нарахування бонусів/знижок (bonus, tariff)."""

    def apply_bonus(cost: float) -> float:
        return cost * (1.0 - discount_percent)
```

```
    return apply_bonus



def with_monthly_cost(
    record: UsageRecord,
    extra_min: float,
    extra_sms: float,
    extra_gb: float,
    final_cost: float,
) -> ProcessedUsageRecord:
    """Повертає новий словник з розрахованими витратами (monthly, cost), не мутуючи оригінал."""
    return {
        **record,
        "extra_minutes_cost": round(extra_min, 2),
        "extra_sms_cost": round(extra_sms, 2),
        "extra_gb_cost": round(extra_gb, 2),
        "monthly_cost": round(final_cost, 2),
    }
```

```
def make_tariff_processor(
    *,
    accept: FilterFn,
    bonus_policy: BonusFn,
) -> Callable[[Iterable[UsageRecord]], dict[str, object]]:
    """Параметризована фабрика обробника тарифів з урахуванням фільтрації (filter, limit)."""

    def process(records: Iterable[UsageRecord]) -> dict[str, object]:
        processed_list: List[ProcessedUsageRecord] = []
        total_revenue = 0.0

        for rec in records:
            if not accept(rec):
                continue

            extra_min = calculate_extra_minutes_cost(rec)
            extra_sms = calculate_extra_sms_cost(rec)
            extra_gb = calculate_extra_gb_cost(rec)
```

```
            subtotal = rec["plan"]["base_fee"] + extra_min + extra_sms + extra_gb
            final_cost = bonus_policy(subtotal)

            new_rec = with_monthly_cost(rec, extra_min, extra_sms, extra_gb, final_cost)
            processed_list.append(new_rec)
            total_revenue += final_cost

        return {
            "count": len(processed_list),
            "revenue": round(total_revenue, 2),
            "records": processed_list,
        }

    return process
```

2. app.py

```
"""Оболонка для роботи з I/O та виклику чистого ядра (Мобільний тариф)."""

from typing import List
```

```
from core import UsageRecord, make_tariff_processor, make_loyalty_bonus_policy



def render_report(result: dict[str, object]) -> None:
    """Функція для відображення результатів (I/O, print)."""
    print("=== ЗВІТ РОЗРАХУНКУ МОБІЛЬНИХ ТАРИФІВ (Варіант 11) ===")
    print(f"Опрацьовано записів: {result['count']}")
    print("-" * 60)


    records = result.get("records", [])
    if isinstance(records, list):
        for rec in records:
            print(
                f"Користувач ID: {rec['user_id']} | "
                f"Абонплата: {rec['plan']['base_fee']} грн | "
                f"Понад ліміт: {rec['extra_minutes_cost'] + rec['extra_sms_cost'] + rec['extra_gb_cost']:.2f} грн | "
                f"Місячна вартість (monthly cost): {rec['monthly_cost']} грн"
            )
```

```
    print("-" * 60)
    print(f"Загальний дохід (Revenue): {result['revenue']} грн\n")



def run() -> None:
    # Початкові дані (mobile, tariff)
    sample_records: List[UsageRecord] = [
        {
            "user_id": 101,
            "minutes_used": 350,
            "sms_used": 40,
            "gb_used": 12.5,
            "plan": {
                "base_fee": 150.0,
                "free_minutes": 300,
                "free_sms": 50,
                "free_gb": 10.0,
                "price_per_min": 1.5,
                "price_per_sms": 0.5,
                "price_per_gb": 20.0,
```

```
    },
  },
  {
    "user_id": 102,
    "minutes_used": 100,
    "sms_used": 10,
    "gb_used": 2.0,
    "plan": {
      "base_fee": 100.0,
      "free_minutes": 200,
      "free_sms": 100,
      "free_gb": 5.0,
      "price_per_min": 1.0,
      "price_per_sms": 0.3,
      "price_per_gb": 15.0,
    },
  },
]
```

```
    # Використання фільтра (filter, limit) та політики бонусів (bonus) через Callable
    accept_active = lambda r: r["minutes_used"] > 0 or r["gb_used"] > 0
    discount_10_percent = make_loyalty_bonus_policy(0.10)


from copy import deepcopy
import pytest
from core import (
    UsageRecord,
    calculate_extra_minutes_cost,
    calculate_extra_gb_cost,
    make_tariff_processor,
    make_loyalty_bonus_policy,
)



@pytest.fixture
def sample_records() -> list[UsageRecord]:
    return [
        {
```

```
            "user_id": 1,
            "minutes_used": 120,
            "sms_used": 15,
            "gb_used": 6.0,
            "plan": {
                "base_fee": 100.0,
                "free_minutes": 100,
                "free_sms": 10,
                "free_gb": 5.0,
                "price_per_min": 2.0,
                "price_per_sms": 1.0,
                "price_per_gb": 10.0,
            },
        }
    ]



def test_referential_transparency(sample_records):
    """Перевірка детермінованості (референтної прозорості)."""
    processor = make_tariff_processor(
```

```
        accept=lambda r: True, bonus_policy=make_loyalty_bonus_policy(0.1)
    )

    r1 = processor(sample_records)
    r2 = processor(sample_records)

    assert r1 == r2


def test_no_mutation(sample_records):
    """Перевірка відсутності мутацій вхідних даних."""
    original = deepcopy(sample_records)
    processor = make_tariff_processor(
        accept=lambda r: True, bonus_policy=lambda x: x
    )

    processor(sample_records)

    assert sample_records == original
```

```
def test_callable_policies(sample_records):
    """Перевірка коректності обчислень з політиками через Callable."""
    processor = make_tariff_processor(
        accept=lambda r: r["user_id"] == 1,
        bonus_policy=make_loyalty_bonus_policy(0.20),
    )

    result = processor(sample_records)
    processed_rec = result["records"][0]

    assert result["count"] == 1
    assert processed_rec["monthly_cost"] == 124.0
    assert processed_rec["extra_minutes_cost"] == 40.0
    assert processed_rec["extra_gb_cost"] == 10.0


def test_extra_cost_calculators(sample_records):
    """Тест чистих функцій підрахунку понад ліміт."""
    rec = sample_records[0]
```

```
    assert calculate_extra_minutes_cost(rec) == 40.0
    assert calculate_extra_gb_cost(rec) == 10.0        accept=accept_active,
        bonus_policy=discount_10_percent,
    )


    result = processor(sample_records)
    render_report(result)



if __name__ == "__main__":
    run()
```

3. tests/test_core.py